

Lesson 10: Advanced SQL and Database Integration

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	WHERE vs HAVING — predict the output
I Do	10–15 min	Subqueries, aggregation, HAVING, transactions
We Do	15–20 min	Build a complex order query together
You Do	5–10 min	Write an aggregation query
Wrap-Up	5 min	Assignment preview & capstone DB goals

Warm-Up (5 min)

WHERE vs HAVING — What's the Difference?

```
-- Query A
SELECT department_id, AVG(salary)
FROM Employees
WHERE salary > 50000
GROUP BY department_id;
```

```
-- Query B
SELECT department_id, AVG(salary)
FROM Employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

What does each query return? Are they the same?

I Do: Aggregation Functions

SQL can summarize data across groups:

```
SELECT department_id,  
       COUNT(*) AS num_employees,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary,  
       AVG(salary) AS avg_salary,  
       SUM(salary) AS total_salary  
FROM Employees  
GROUP BY department_id;
```

I Do: Aggregation Functions (Cont.)

Function	What It Does
<code>COUNT(*)</code>	Number of rows in each group
<code>MIN()</code>	Smallest value
<code>MAX()</code>	Largest value
<code>AVG()</code>	Mean value
<code>SUM()</code>	Total of all values

Every non-aggregated column in `SELECT` must be in `GROUP BY`.

I Do: HAVING — Filter After Grouping

WHERE filters rows. **HAVING** filters groups.

```
-- Find departments with more than 5 employees
SELECT department_id, COUNT(*) AS emp_count
FROM Employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

SQL execution order:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

1. **WHERE** runs first — filters individual rows
2. **GROUP BY** creates the groups
3. **HAVING** filters groups based on aggregate values
4. **SELECT** picks the columns to return

I Do: Subqueries

A subquery is a query inside another query:

```
-- Find the highest-paid employee in each department
SELECT name, salary, department_id
FROM Employees AS e
WHERE salary = (
    SELECT MAX(salary)
    FROM Employees
    WHERE department_id = e.department_id
);
```

I Do: Subqueries (Cont.)

How it works:

1. The **inner query** runs for each row of the outer query
2. It finds `MAX(salary)` for that department
3. The **outer query** keeps rows where salary matches the max

Subqueries let you ask "find rows where this value equals the result of another query."

I Do: Subqueries in FROM Clause

Subqueries can also act as temporary tables:

```
SELECT c.customer_name, AVG(sub.total_price) AS avg_order
FROM customers c
LEFT JOIN (
    SELECT o.customer_id AS customer_id_b,
           SUM(p.price * li.quantity) AS total_price
    FROM orders o
    JOIN line_items li ON o.order_id = li.order_id
    JOIN products p ON li.product_id = p.product_id
    GROUP BY o.order_id
) AS sub ON c.customer_id = sub.customer_id_b
GROUP BY c.customer_id;
```

I Do: Subqueries in FROM Clause (Cont.)

Key details:

- The subquery creates a temporary result set (aliased as `sub`)
- The outer query JOINS against it like a regular table
- Use `AS` to alias columns and avoid name collisions

I Do: Transactions and Rollbacks

Transactions ensure "all or nothing" consistency:

```
conn = sqlite3.connect("company.db")
cursor = conn.cursor()

try:
    cursor.execute("INSERT INTO orders ...")
    cursor.execute("INSERT INTO line_items ...")
    cursor.execute("INSERT INTO line_items ...")
    conn.commit()    # All succeed → save everything
except Exception as e:
    conn.rollback()  # Any failure → undo everything
    print(f"Error: {e}")
```

I Do: Transactions and Rollbacks (Cont.)

Why transactions matter:

- Without them, a failed INSERT could leave partial data
- Example: an order with no line items, or line items with no order
- `commit()` saves all changes; `rollback()` undoes them all

I Do: SQL Injection and Parameterized Queries

Never build SQL with string concatenation:

```
# DANGEROUS – SQL injection vulnerability!
cursor.execute(f"SELECT * FROM Users WHERE name = '{user_input}'")

# SAFE – parameterized query
cursor.execute("SELECT * FROM Users WHERE name = ?", (user_input,))
```

What could go wrong?

If `user_input` is `' ; DROP TABLE Users; --`, the dangerous version runs:

```
SELECT * FROM Users WHERE name = ' ; DROP TABLE Users; --'
```

Always use `?` placeholders. Let the database sanitize the input.

I Do: Window Functions and Indexing

Window functions — analytics without collapsing rows:

```
SELECT name, salary, department_id,  
       RANK() OVER (  
         PARTITION BY department_id  
         ORDER BY salary DESC  
       ) AS salary_rank  
FROM Employees;
```

Unlike `GROUP BY`, window functions keep every row and add a computed column.

Indexing — speed up frequent queries:

```
CREATE INDEX idx_dept ON Employees(department_id);
```

- Indexes make `WHERE` and `JOIN` on that column faster
- Trade-off: indexes slow down `INSERT` / `UPDATE` slightly

We Do: Build a Complex Order Query

Assignment Task 1 asks: Find the total price of each of the first 5 orders.

Step 1: Think about what tables we need:

```
orders → line_items → products
  ↓           ↓           ↓
order_id  quantity  price
```

Each order has multiple line items. Each line item references a product. Total price = `SUM(price * quantity)` per order.

Step 2: Build it piece by piece in `sqlcommand`:

We Do: Build a Complex Order Query (Cont.)

```
SELECT o.order_id, SUM(p.price * li.quantity) AS total
FROM orders o
JOIN line_items li ON o.order_id = li.order_id
JOIN products p ON li.product_id = p.product_id
GROUP BY o.order_id
ORDER BY o.order_id
LIMIT 5;
```


We Do: Wrap It in Python

Step 3: Put the working query into a Python script:

```
import sqlite3

with sqlite3.connect("../db/lesson.db") as conn:
    cursor = conn.cursor()

    cursor.execute("""
        SELECT o.order_id,
               SUM(p.price * li.quantity) AS total
        FROM orders o
        JOIN line_items li ON o.order_id = li.order_id
        JOIN products p ON li.product_id = p.product_id
        GROUP BY o.order_id
        ORDER BY o.order_id
        LIMIT 5
    """)

    for row in cursor.fetchall():
        print(f"Order {row[0]}: ${row[1]:.2f}")
```

We Do: Add a HAVING Filter

Step 4: What if we only want orders over \$100?

```
SELECT o.order_id,  
       SUM(p.price * li.quantity) AS total  
FROM orders o  
JOIN line_items li ON o.order_id = li.order_id  
JOIN products p ON li.product_id = p.product_id  
GROUP BY o.order_id  
HAVING total > 100  
ORDER BY total DESC  
LIMIT 5;
```

Discussion: Why can't we use `WHERE total > 100`?

Because `total` doesn't exist until after `GROUP BY` runs. `HAVING` filters after aggregation; `WHERE` filters before.

We Do: Build a Transaction

Step 5: Create a new order with multiple line items:

```
try:
    # Look up IDs we need
    cursor.execute(
        "SELECT customer_id FROM customers WHERE customer_name = ?",
        ("Perez and Sons",))
    cust_id = cursor.fetchone()[0]

    cursor.execute(
        "SELECT employee_id FROM employees WHERE last_name = ?",
        ("Harris",))
    emp_id = cursor.fetchone()[0]

    # Create the order (get its ID back)
    cursor.execute(
        "INSERT INTO orders (customer_id, employee_id) VALUES (?,?) RETURNING order_id",
        (cust_id, emp_id))
    order_id = cursor.fetchone()[0]

    # ... add line items using order_id ...
    conn.commit()
except:
```

You Do: Aggregation Query Challenge (5 min)

Using `lesson.db`, write a query that finds:

The total number of orders per customer, showing only customers with more than 3 orders. Return the customer name and order count, sorted by count descending.

Hints:

- JOIN `customers` with `orders`
- GROUP BY `customer`
- COUNT(*) for order count
- HAVING to filter
- ORDER BY to sort

Assignment Preview

Task	What You'll Build
1	Total price of first 5 orders (JOIN + GROUP BY + SUM)
2	Average order price per customer (subquery + LEFT JOIN)
3	Create a new order with 5 line items (transaction)
4	Employees with more than 5 orders (HAVING)
5	Finalize capstone SQLite database

Assignment Tips

General approach: Test every query in `sqlcommand.py` before writing Python code.

Task 2 (subqueries): Build the inner query first and verify it returns order totals. Then wrap it in the outer query.

Task 3 (transactions):

- Use `RETURNING order_id` on the INSERT to get the auto-generated ID
- Look up the 5 cheapest products: `ORDER BY price LIMIT 5`
- Wrap all INSERTs in `try/except` with `rollback()`

Task 4 (HAVING): Remember — `HAVING` filters groups, `WHERE` filters rows.

Wrap-Up

Today we covered:

- Aggregation functions: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- `GROUP BY` + `HAVING` for filtering aggregated results
- Subqueries — queries inside queries for dynamic filtering
- Transactions with `commit()` and `rollback()`
- Parameterized queries to prevent SQL injection
- Window functions and indexing (introduction)

Quick check: When do you use `HAVING` instead of `WHERE`?

Getting Help This Week

- Use `sqlcommand.py` to test queries interactively before coding
- Build complex queries **one piece at a time** — start simple, add clauses
- The `RETURNING` clause is essential for Task 3 — don't skip it
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** — this is a tough assignment!

Next week: Advanced Data Visualization — Plotly, Streamlit, and your final dashboard!

SQL Leveled Up! 🚀

From basic queries to real-world data operations.

See you next session!